In [11]:
```
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import matplotlib.pyplot as plt
from scipy.io import loadmat
import numpy as np
import os
import torch
import torch.nn as nn
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['SimHei']
matplotlib.rcParams['axes.unicode_minus'] = False
matplotlib.use('Agg')
```

导入相关包，解决中文输入问题

In [12]:
```
BATCH_SIZE = 64
EPOCHS = 20
LEARNING_RATE = 0.001
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATA_PATH = "./data"
MODEL_SAVE_PATH = "./svhn_cnn.pt"
```

配置相关参数

In [13]:
```
class SVHNDataset(Dataset):
    def __init__(self, data_path, transform=None, is_train=True):
        if is_train:
            mat = loadmat(os.path.join(data_path, "train_32x32.mat"))
        else:
            mat = loadmat(os.path.join(data_path, "test_32x32.mat"))

        self.images = np.transpose(mat['X'], (3, 2, 0, 1)).astype(np.float32)
        self.labels = mat['y'].flatten()
        self.labels[self.labels == 10] = 0
        self.transform = transform

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        img = self.images[idx]
        img = np.transpose(img, (1, 2, 0))
        label = self.labels[idx]

        if self.transform:
            img = self.transform(img)

        return img, label
```

数据预处理

In [14]:
```
train_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

数据增强

In [15]:
```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

构建相应的CNN模型

In [16]:
```
def train_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    return total_loss / len(loader), correct / total


def test_epoch(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    all_preds = []
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            outputs = model(imgs)
```

```
            loss = criterion(outputs, labels)

            total_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
            all_preds.extend(predicted.cpu().numpy())
    return total_loss / len(loader), correct / total, all_preds
```

训练函数以及测试函数

In [17]:
```
def plot_training_curve(train_accs, test_accs, train_losses, test_losses):
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(train_accs, label='Train Accuracy')
    plt.plot(test_accs, label='Test Accuracy')
    plt.title('Accuracy vs Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(train_losses, label='Train Loss')
    plt.plot(test_losses, label='Test Loss')
    plt.title('Loss vs Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.tight_layout()
    plt.savefig('training_curves.png')
    plt.close()


def plot_test_predictions(dataset, predictions):
    indices = np.random.choice(len(dataset), 9)
    plt.figure(figsize=(9, 9))
    for i, idx in enumerate(indices):
        img, label = dataset[idx]
        pred = predictions[idx]

        img = img.numpy().transpose((1, 2, 0))
        img = img * 0.5 + 0.5
        img = np.clip(img, 0, 1)

        plt.subplot(3, 3, i + 1)
        plt.imshow(img)
        color = "green" if label == pred else "red"
        plt.title(f"True: {label}\nPred: {pred}", color=color)
        plt.axis("off")
    plt.tight_layout()
    plt.savefig('test_predictions.png')
    plt.close()
```

可视化函数

In [18]:
```
if __name__ == '__main__':
    # 1. 加载数据
    train_dataset = SVHNDataset(DATA_PATH, train_transform, is_train=True)
```

```
    test_dataset = SVHNDataset(DATA_PATH, test_transform, is_train=False)

    train_loader = DataLoader(train_dataset, BATCH_SIZE, shuffle=True, num_worke
    test_loader = DataLoader(test_dataset, BATCH_SIZE, shuffle=False, num_worker

    # 2. 初始化模型
    model = SimpleCNN().to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

    # 3. 开始训练
    print("=" * 50)
    print("开始训练 SVHN 分类模型")
    print("=" * 50)
    train_accs, test_accs = [], []
    train_losses, test_losses = [], []

    for epoch in range(EPOCHS):
        train_loss, train_acc = train_epoch(model, train_loader, criterion, opti
        test_loss, test_acc, _ = test_epoch(model, test_loader, criterion)

        train_accs.append(train_acc)
        test_accs.append(test_acc)
        train_losses.append(train_loss)
        test_losses.append(test_loss)

        print(f"Epoch [{epoch + 1}/{EPOCHS}]")
        print(f"Train Loss: {train_loss:.4f} | Acc: {train_acc:.4f}")
        print(f"Test  Loss: {test_loss:.4f} | Acc: {test_acc:.4f}\n")

    # 4. 保存模型
    torch.save(model.state_dict(), MODEL_SAVE_PATH)
    print(f" 模型已保存至: {MODEL_SAVE_PATH}")

    # 5. 绘制训练曲线
    plot_training_curve(train_accs, test_accs, train_losses, test_losses)
    print(" 训练曲线已保存: training_curves.png")

    # 6. 最终测试集评估（完整测试）
    final_test_loss, final_test_acc, test_predictions = test_epoch(model, test_l
    print("=" * 50)
    print("最终测试集结果: ")
    print(f"测试损失: {final_test_loss:.4f}")
    print(f"测试准确率: {final_test_acc:.4f} ({final_test_acc * 100:.2f}%)")
    print(f"测试集样本数: {len(test_dataset)}")
    print("=" * 50)

    # 7. 绘制测试预测图
    plot_test_predictions(test_dataset, test_predictions)
    print(" 测试预测图已保存: test_predictions.png")
```

```
==================================================
开始训练 SVHN 分类模型
==================================================
Epoch [1/20]
Train Loss: 1.9489 | Acc: 0.4367
Test  Loss: 1.0647 | Acc: 0.7054

Epoch [2/20]
Train Loss: 1.1275 | Acc: 0.6444
Test  Loss: 0.9494 | Acc: 0.7339

Epoch [3/20]
Train Loss: 0.9104 | Acc: 0.7152
Test  Loss: 0.6924 | Acc: 0.7988

Epoch [4/20]
Train Loss: 0.7366 | Acc: 0.7743
Test  Loss: 0.5841 | Acc: 0.8403

Epoch [5/20]
Train Loss: 0.6468 | Acc: 0.8051
Test  Loss: 0.5421 | Acc: 0.8490

Epoch [6/20]
Train Loss: 0.5978 | Acc: 0.8203
Test  Loss: 0.5075 | Acc: 0.8583

Epoch [7/20]
Train Loss: 0.5574 | Acc: 0.8335
Test  Loss: 0.5257 | Acc: 0.8522

Epoch [8/20]
Train Loss: 0.5376 | Acc: 0.8388
Test  Loss: 0.4474 | Acc: 0.8768

Epoch [9/20]
Train Loss: 0.5075 | Acc: 0.8464
Test  Loss: 0.4491 | Acc: 0.8806

Epoch [10/20]
Train Loss: 0.4945 | Acc: 0.8528
Test  Loss: 0.4919 | Acc: 0.8674

Epoch [11/20]
Train Loss: 0.4821 | Acc: 0.8564
Test  Loss: 0.4193 | Acc: 0.8875

Epoch [12/20]
Train Loss: 0.4703 | Acc: 0.8607
Test  Loss: 0.4207 | Acc: 0.8848

Epoch [13/20]
Train Loss: 0.4603 | Acc: 0.8623
Test  Loss: 0.4239 | Acc: 0.8833

Epoch [14/20]
Train Loss: 0.4487 | Acc: 0.8664
Test  Loss: 0.3995 | Acc: 0.8944

Epoch [15/20]
```

```
Train Loss: 0.4356 | Acc: 0.8702
Test  Loss: 0.4053 | Acc: 0.8930

Epoch [16/20]
Train Loss: 0.4302 | Acc: 0.8727
Test  Loss: 0.4199 | Acc: 0.8861

Epoch [17/20]
Train Loss: 0.4173 | Acc: 0.8762
Test  Loss: 0.3971 | Acc: 0.8935

Epoch [18/20]
Train Loss: 0.4104 | Acc: 0.8765
Test  Loss: 0.4247 | Acc: 0.8903

Epoch [19/20]
Train Loss: 0.4099 | Acc: 0.8790
Test  Loss: 0.4164 | Acc: 0.8864

Epoch [20/20]
Train Loss: 0.3961 | Acc: 0.8833
Test  Loss: 0.4287 | Acc: 0.8856

 模型已保存至: ./svhn_cnn.pt
 训练曲线已保存: training_curves.png
==================================================
最终测试集结果:
测试损失: 0.4287
测试准确率: 0.8856 (88.56%)
测试集样本数: 26032
==================================================
 测试预测图已保存: test_predictions.png
```

主程序